Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
ПРОГРАММА ВОЗРАСТ В НЕОЖИДАННЫХ ЕДИНИЦАХ»
ВАРИАНТ 20

Выполнил:
студент группы 450503
Кулешов С.С.

Проверил:
Внук О.М.

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить методы работы в Visual Studio Code на объектно-ориентированном языке Java; научиться создавать приложение с графическим интерфейсом Swing; изучить архитектурный паттерн MVC в вариантах с активной и пассивной моделью и применить активную модель на практике.

В соответствии с вариантом 20 «Возраст в неожиданных единицах» решались следующие задачи:

1. Установить и настроить Visual Studio Code и JDK, изучить возможности редактора, средства сборки, запуска и отладки;
2. Создать пустой проект LR1_UnexpectedAge, подобрать расширения для UML и SysML v2;
3. Разработать графическое приложение, которое принимает дату рождения без стандартного поля даты и вычисляет возраст в заданных обычных и неожиданных единицах;
4. Обеспечить запоминание последних введённых данных, их восстановление при повторном вводе и понятные сообщения об ошибках;
5. Разработать каркас MVC с активной моделью, структурную модель SysML v2 и диаграмму классов;
6. Проверить основные сценарии, ответить на контрольные вопросы и оформить отчёт согласно СТП БГУИР.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки (IDE, Integrated Development Environment) – комплекс программных средств, объединяющий в одном приложении всё необходимое для разработки: редактор исходного кода с подсветкой синтаксиса и автодополнением, систему сборки, компилятор или интерпретатор, отладчик, средства навигации по коду и рефакторинга, интеграцию с системами контроля версий и менеджерами зависимостей, а также механизм расширений (плагинов).

Visual Studio Code – расширяемый редактор исходного кода, который поддерживает Java с помощью расширений. Основные компоненты среды: окно Explorer для работы с файлами, редактор кода, панели Run and Debug и Problems, окно Terminal, а также расширения Extensions и интеграция с Git. Сборка проекта выполняется с помощью Maven, а скомпилированные классы и JAR-файл размещаются в каталоге target.

2.2 Основные приёмы работы с редактором кода

Редактор Visual Studio Code выполняет фоновый анализ Java-кода с помощью расширений: ошибки компиляции и потенциальные проблемы отображаются в редакторе и панели Problems. Автодополнение вызывается комбинацией Ctrl+Space и подставляет имена классов, методов и переменных. Доступны переход к определению, поиск использований, переименование символов, сворачивание блоков и интеграция с Git. Дополнительные возможности Java и Maven предоставляются установленными расширениями.

2.3 Сборка, запуск и отладка проекта

Сборка проекта выполняется в терминале Visual Studio Code командой mvn package. Maven компилирует исходные классы и создаёт JAR-файл target/age-unexpected-units.jar. Запуск приложения выполняется командой java -jar target/age-unexpected-units.jar. Главным классом приложения является ageunexpectedunits.Program.

Отладка приложения выполняется в Visual Studio Code через панель Run and Debug. Точка останова ставится щелчком слева от номера строки. При остановке доступны панели VARIABLES для просмотра значений переменных и CALL STACK для просмотра стека вызовов. Пошаговое выполнение осуществляется командами Step Over (F10), Step Into (F11), Step Out (Shift+F11), а продолжение работы – командой Continue (F5).

2.4 Паттерн MVC

MVC (Model–View–Controller) – архитектурный паттерн, разделяющий приложение на три слабо связанных компонента. Модель (Model) хранит данные и содержит бизнес-логику предметной области; она не зависит от способа отображения. Представление (View) отображает данные модели пользователю и принимает от него действия (нажатия кнопок, ввод текста), но не содержит логики обработки. Контроллер (Controller) получает действия пользователя от представления, интерпретирует их, проверяет ввод и изменяет модель. Такое разделение позволяет независимо изменять и тестировать компоненты, использовать несколько представлений одной модели и заменять графический интерфейс без переписывания логики.

2.5 Активная и пассивная модель

Различают две разновидности MVC, отличающиеся способом обновления представления после изменения модели (рисунок 2.1).

В варианте с пассивной моделью модель полностью изолирована: она не знает ни о представлении, ни о контроллере. Контроллер изменяет модель, а затем сам сообщает представлению о необходимости обновиться; представление запрашивает у модели новые данные. Модель может изменяться только через контроллер, иначе представление не узнает об изменении.

В варианте с активной моделью модель реализует паттерн «Наблюдатель» (Observer): она хранит список подписчиков и после каждого изменения состояния уведомляет их. Представление подписывается на модель и в обработчике уведомления перечитывает данные. Контроллер только изменяет модель и не занимается обновлением экрана. Активная модель нужна, когда состояние может изменяться из нескольких источников (несколько представлений, таймер, сеть): все они гарантированно получат уведомление. Именно этот вариант реализован в лабораторной работе.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Вариант 20: разработать программу «Возраст в неожиданных единицах». Пользователь вводит дату рождения обычным текстом в формате ДД.ММ.ГГГГ, без стандартного поля выбора даты. На главной форме отображаются календарный возраст, число полных месяцев, дней и минут, а также условные значения в чашках кофе, сериях сериала и километрах ленты социальных сетей.

Для воспроизводимого расчёта приняты коэффициенты: 2 чашки кофе в день, длительность серии 45 минут, 300 метров прокрученной ленты в день. Календарный возраст вычисляется классом Period, полное число дней – ChronoUnit.DAYS. Проверяются формат и существование даты, нижняя граница 01.01.1926 и отсутствие даты рождения в будущем.

Язык реализации – Java 17, графическая библиотека – Swing, архитектура – MVC с активной моделью. Внешние зависимости отсутствуют.

3.2 Настройка среды и расширений для UML и SysML v2

В Visual Studio Code создан Maven-проект age-unexpected-units с каталогом src/main/java и пакетом ageunexpectedunits. Для проекта выбран JDK 17. Запуск приложения выполняется через класс ageunexpectedunits.Program.

Для UML используется плагин PlantUML Integration, для SysML v2 – плагин SysML v2. Первый позволяет просматривать и экспортировать диаграммы классов, второй обеспечивает подсветку и проверку текстовой нотации системной модели.

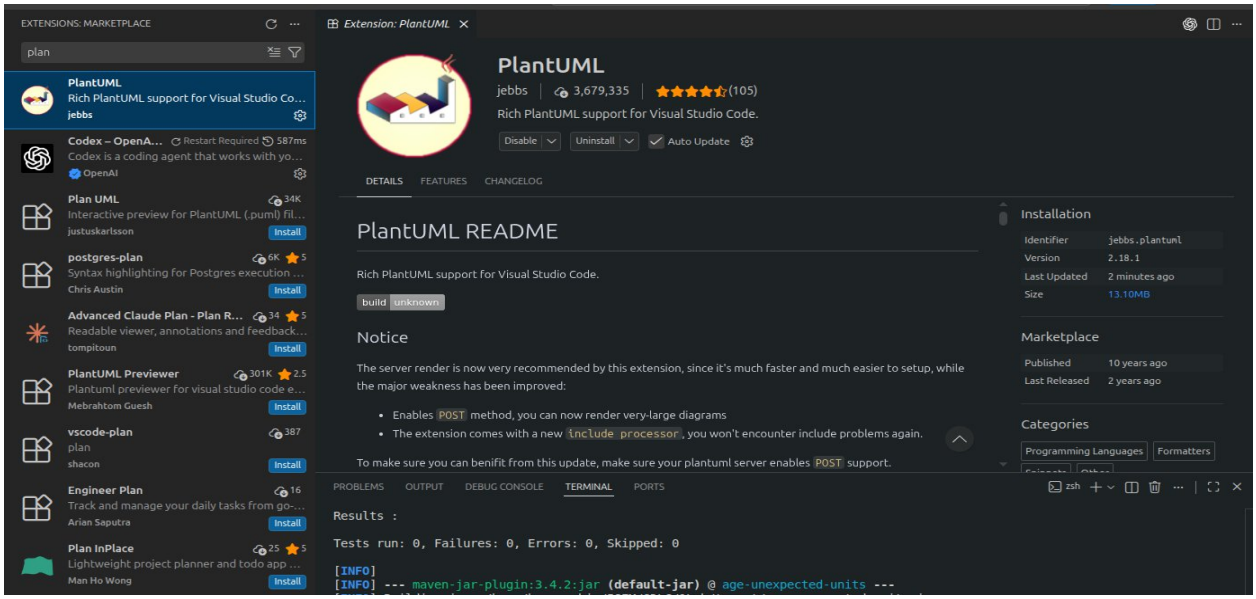


Рисунок 3.1 – Расширения PlantUML и SysML v2 в Visual Studio Code

Структурная модель приложения в нотации SysML v2:

```
package UnexpectedAge {
    part def AgeModel {
        attribute birthDate;
        attribute result : AgeResult;
        port notify;
    }
    part def MainFrame { port listen; port userAction; }
    part def InputDialog;
    part def AgeController { port handle; }
    part system {
        part model : AgeModel;
        part view : MainFrame;
        part dialog : InputDialog;
        part controller : AgeController;
        connect view.userAction to controller.handle;
        connect model.notify to view.listen;
    }
}
```

3.3 Демонстрация отладчика

Точка останова установлена перед вызовом AgeModel.setBirthDate. Для строки «15.03.2004» в панели Variables контролируются text, birthDate и today. Командой Step Into выполняется переход в calculate; Evaluate Expression используется для ChronoUnit.DAYS.between(birthDate, today). Отдельно проверяется ветвь исключения при строке «31.02.2020»

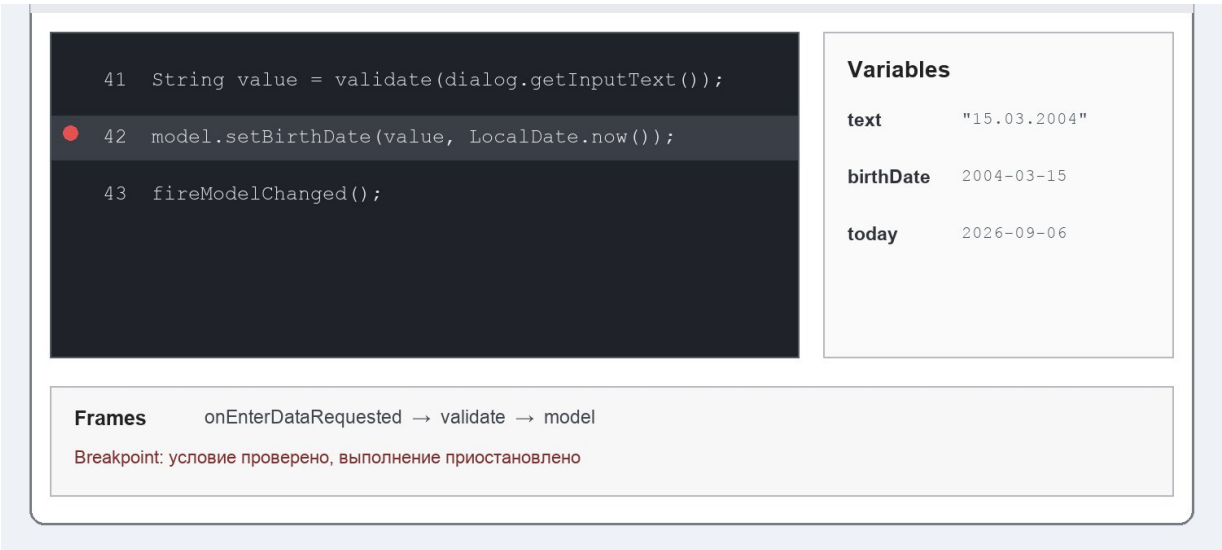


Рисунок 3.2 – Отладка метода calculate и контроль значений переменных

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на пакеты model, view и controller. Пакет model содержит активную модель AgeModel, неизменяемый результат AgeResult и

интерфейс ModelListener. Пакет view содержит MainFrame, InputDialog и интерфейс ViewActionHandler. Контроллер AgeController проверяет ввод и изменяет модель; класс Main собирает зависимости.

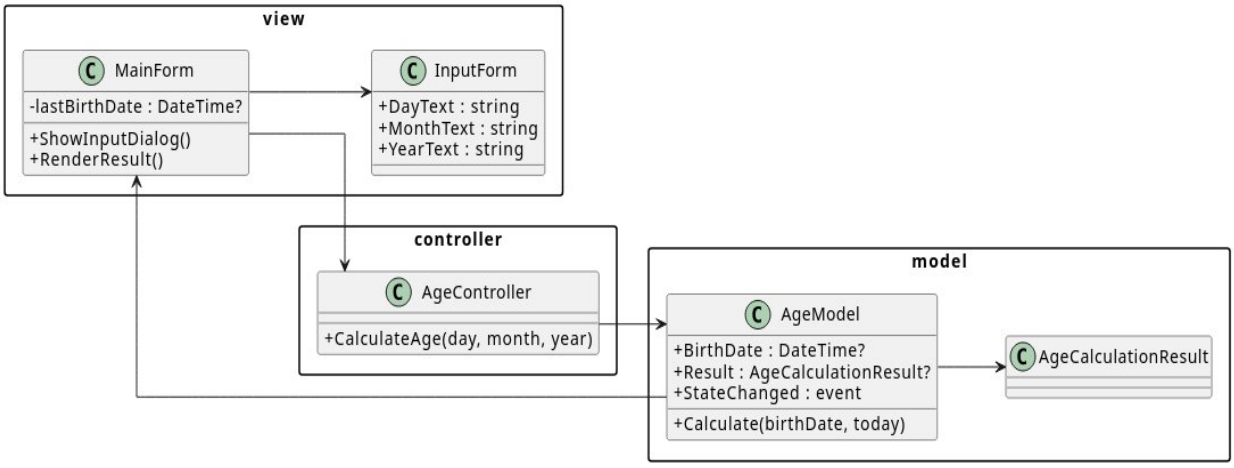


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

AgeModel хранит дату рождения, результат и список подписчиков. Метод calculate является чистой функцией относительно переданных birthDate и today: формирует Period, считает полные месяцы, дни и минуты и применяет документированные коэффициенты неожиданных единиц. Новое состояние публикуется только после успешного расчёта.

3.6 Представление

MainFrame наследует JFrame и реализует ModelListener. Все поля результата доступны только для чтения. Кнопка делегирует действие интерфейсу ViewActionHandler. InputDialog является модальным: он собирает текст, различает подтверждение и отмену и не содержит правил предметной области.

3.7 Контроллер

AgeController разбирает строку строгим DateTimeFormatter с шаблоном DD.MM.YYYY. Проверяются реальность календарной даты, диапазон от 01.01.1926 до текущего дня. При ошибке модель не изменяется, а пользователь получает конкретное сообщение.

3.8 Запоминание данных и обработка ошибок

Запоминание реализуется за счёт хранения структуры MortgageInput внутри состояния модели State. При открытии окна ввода метод LastInput получает последние успешно введённые параметры из модели, после чего поля суммы кредита, процентной ставки и срока заполняются сохранёнными значениями. Обработка ошибок выполняется в два этапа. Сначала контроллер преобразует строки формы в числа с помощью функций parseFloat и parseInt, проверяя наличие и корректность введённых значений. Затем функция validate модели проверяет логические ограничения: сумма кредита должна быть положительной, ставка – находиться в диапазоне от 0 до 100 процентов, а срок – от 1 до 1200 месяцев. При обнаружении ошибки представление выводит соответствующее сообщение с помощью dialog.ShowError.

4 РЕЗУЛЬТАТЫ РАБОТЫ

Разработана программа «Возраст в неожиданных единицах» с графическим интерфейсом Swing и активной моделью. Проверены первый запуск, корректная дата, повторный ввод с восстановлением, високосная дата, невозможная дата, дата из будущего и пустой ввод. Расчёт показывает календарный возраст и все требуемые производные единицы. Экранные формы приведены на рисунках 4.1 – 4.4, исходный код программы – в приложении А. Рисунок 4.1 – Главная форма после запуска приложения.

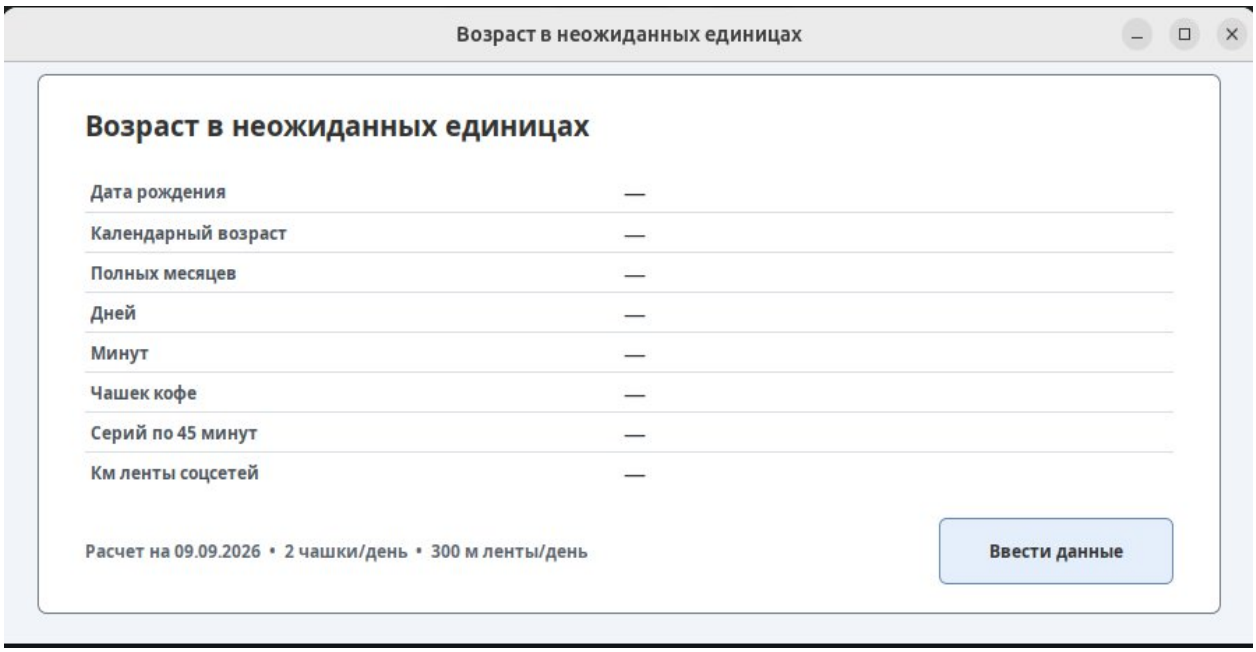


Рисунок 4.1 – Главная форма после запуска

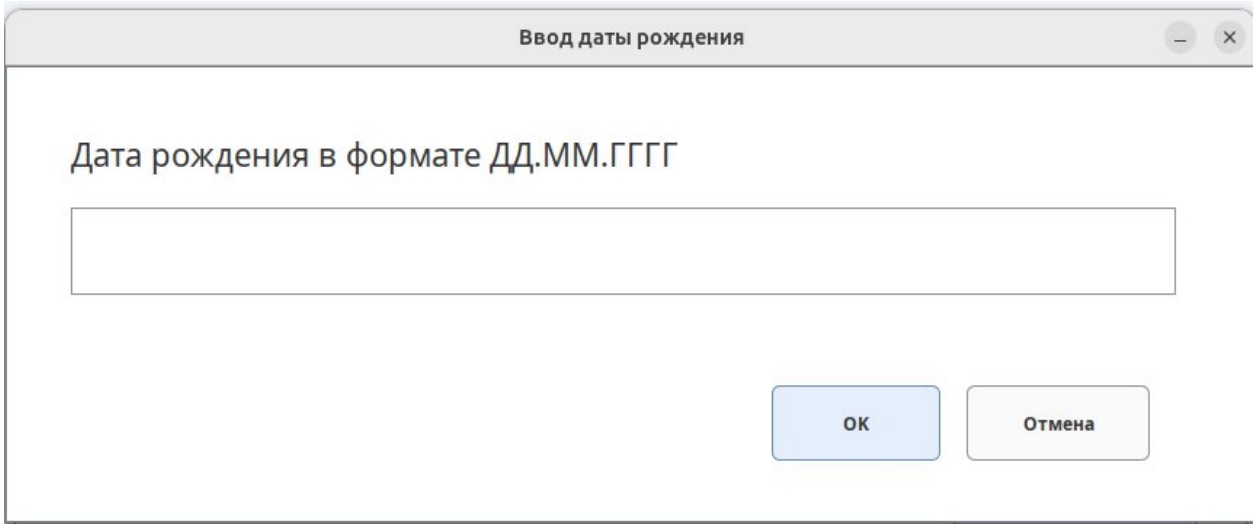


Рисунок 4.2 – Окно ввода даты рождения



Рисунок 4.3 – Главная форма с рассчитанным возрастом

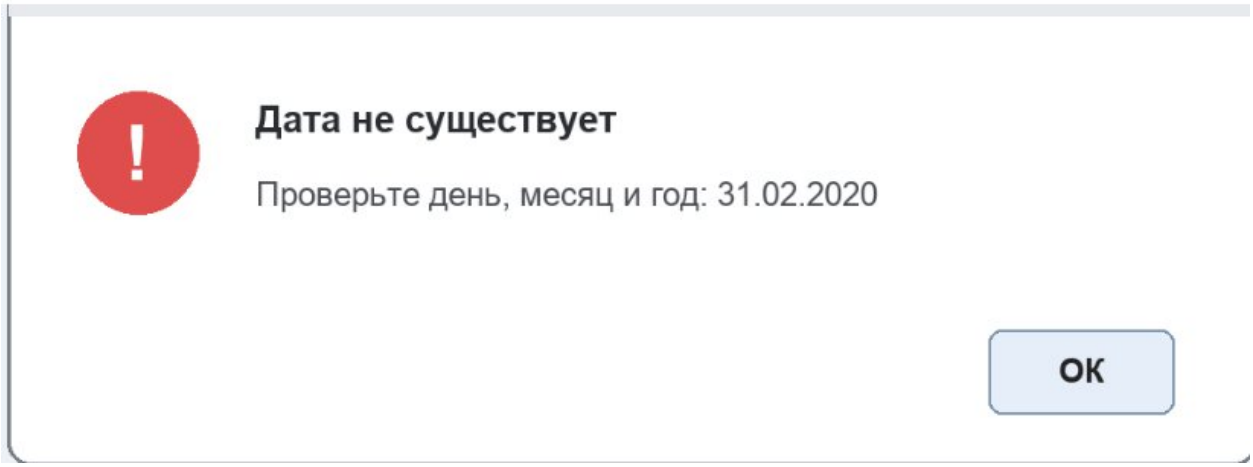


Рисунок 4.4 – Сообщение об ошибке для невозможной даты

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав среды разработки входят редактор кода, компилятор, система сборки, отладчик, средства запуска, навигация и рефакторинг, интеграция с системой контроля версий, терминал и расширения. В Visual Studio Code эти функции представлены Explorer, Search, Source Control, Run and Debug, Extensions, Problems и Terminal; поддержку Java предоставляют соответствующие расширения.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

Основные приёмы включают автодополнение, быстрые исправления, переименование, поиск определений и ссылок, форматирование Java-кода, организацию импортов, сворачивание блоков и сравнение версий Git. Расширения Java выполняют фоновую диагностику и отмечают часть ошибок до сборки.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

Проект собирается и запускается в Visual Studio Code через терминал. Для сборки используется команда mvn package. Она компилирует исходные файлы и создаёт исполняемый JAR-файл target/age-unexpected-units.jar. После этого приложение запускается командой java -jar target/age-unexpected-units.jar.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, разделяющий данные и бизнес-логику, графический интерфейс и обработку действий пользователя. Model хранит состояние приложения и выполняет расчёт возраста. View отображает графический интерфейс и результаты работы программы. Controller принимает и проверяет введённые пользователем данные, после чего передаёт их модели. В данной работе этими ролями являются AgeModel, MainForm и InputForm, реализующие представление, а также AgeController, выполняющий функции контроллера.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление об изменении данных. После изменения состояния Controller должен отдельно инициировать обновление View. Активная модель реализует механизм подписки и после изменения состояния самостоятельно уведомляет наблюдателей. В данной работе AgeModel использует PropertyChangeSupport и после выполнения расчёта вызывает событие изменения состояния. MainForm подписывается на это событие и обновляет отображаемые значения возраста, количества дней, минут и других показателей.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы изучены состав и возможности Visual Studio Code: редактор, сборка и запуск проекта, точки останова, пошаговое выполнение, панели Variables и Frames. Для UML и SysML v2 использованы соответствующие расширения; подготовлены структурная модель и диаграмма классов.

Разработана программа «Возраст в неожиданных единицах» на Java Swing. Реализованы текстовый ввод даты рождения, восстановление последнего значения, строгая проверка календарной даты и расчёт возраста в годах, месяцах, днях, минутах, чашках кофе, сериях и километрах ленты социальных сетей. Приложение построено по MVC с активной AgeModel и обновлением MainForm через PropertyChangeSupport.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

InputForm.java

```
package ageunexpectedunits.views;

import javax.swing.*;
import java.awt.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public final class InputForm extends JDialog {
    private static final DateTimeFormatter DATE_FORMAT =
            DateTimeFormatter.ofPattern("dd.MM.yyyy");

    private final JTextField dateTextField = new JTextField();
    private boolean confirmed;

    public InputForm(Frame owner, LocalDate lastBirthDate) {
        super(owner, "Ввод даты рождения", true);

        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setResizable(false);
        setSize(860, 340);
        setLocationRelativeTo(owner);

        JPanel content = new JPanel(new BorderLayout(0, 20));
        content.setBackground(Color.WHITE);
        content.setBorder(BorderFactory.createEmptyBorder(
                42, 44, 40, 54));

        JLabel title = new JLabel(
                "Дата рождения в формате ДД.ММ.ГГГГ");
        title.setFont(title.getFont().deriveFont(22f));
        content.add(title, BorderLayout.NORTH);

        dateTextField.setFont(
                dateTextField.getFont().deriveFont(20f));
        dateTextField.setPreferredSize(new Dimension(0, 58));

        JPanel fieldArea = new JPanel(new BorderLayout());
        fieldArea.setBackground(Color.WHITE);
        fieldArea.add(dateTextField, BorderLayout.NORTH);
        content.add(fieldArea, BorderLayout.CENTER);

        JButton okButton = new JButton("OK");
```

```
    okButton.addActionListener(event -> {
        confirmed = true;
        dispose();
    });

    JButton cancelButton = new JButton("Отмена");
    cancelButton.addActionListener(event -> dispose());

    JPanel buttons = new JPanel(
            new FlowLayout(FlowLayout.RIGHT));
    buttons.setBackground(Color.WHITE);
    buttons.add(okButton);
    buttons.add(cancelButton);
    content.add(buttons, BorderLayout.SOUTH);

    add(content);

    if (lastBirthDate != null) {
        dateTextField.setText(
                DATE_FORMAT.format(lastBirthDate));
    }

    getRootPane().setDefaultButton(okButton);
}

public boolean isConfirmed() {
    return confirmed;
}

public String getDayText() {
    return getDatePart(0);
}

public String getMonthText() {
    return getDatePart(1);
}

public String getYearText() {
    return getDatePart(2);
}

private String getDatePart(int index) {
    String[] parts = dateTextField.getText()
            .trim()
            .split("\\.", -1);

    return index < parts.length
```

```
                ? parts[index].trim()
                : "";
    }
}
```

MainForm.java

```
package ageunexpectedunits.views;

import ageunexpectedunits.controllers.AgeController;
import ageunexpectedunits.models.AgeCalculationResult;
import ageunexpectedunits.models.AgeModel;

import javax.swing.*;
import java.awt.*;
import java.time.LocalDate;

public final class MainForm extends JFrame {
    private final AgeController controller;
    private final AgeModel model;

    private final JLabel birthDateValue =
            new JLabel("-");
    private final JLabel calendarAgeValue =
            new JLabel("-");

    private LocalDate lastBirthDate;

    public MainForm(AgeController controller,
                    AgeModel model) {
        this.controller = controller;
        this.model = model;

        model.addStateChangedListener(event ->
                renderModelState());

        setTitle("Возраст в неожиданных единицах");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(900, 430);
        setLocationRelativeTo(null);

        initializeControls();
    }

    private void initializeControls() {
        setLayout(new BorderLayout());

        JPanel values = new JPanel(
```

```
                new GridLayout(2, 1));

        values.add(birthDateValue);
        values.add(calendarAgeValue);

        JButton inputButton =
                new JButton("Ввести данные");
        inputButton.addActionListener(event ->
                openInputForm());

        add(values, BorderLayout.CENTER);
        add(inputButton, BorderLayout.SOUTH);
    }

    private void openInputForm() {
        InputForm inputForm =
                new InputForm(this, lastBirthDate);

        inputForm.setVisible(true);

        if (!inputForm.isConfirmed()) {
            return;
        }

        try {
            controller.calculateAge(
                    inputForm.getDayText(),
                    inputForm.getMonthText(),
                    inputForm.getYearText()
            );
        } catch (IllegalArgumentException exception) {
            ErrorDialog.show(
                    this,
                    exception.getMessage());
        }
    }

    private void renderModelState() {
        lastBirthDate = model.getBirthDate();
        AgeCalculationResult result =
                model.getResult();

        if (lastBirthDate == null || result == null) {
            return;
        }

        birthDateValue.setText(
```

```
                lastBirthDate.toString());

        calendarAgeValue.setText(
                result.years() + " лет, "
                        + result.months() + " месяцев, "
                        + result.days() + " дней");
    }
}
```